A PRELIMINARY MINI PROJECT REPORT ON

MediConnect: Distributed Pharmacy Procurement and Multi-Distributor Medicine Supply Network

SUBMITTED TOWARDS THE PARTIAL FULFILLMENT OF THE REQUIREMENTS OF

BACHELOR OF TECHNOLOGY (B. Tech.)

Academic Year: 2026–27

By:

Name	PRN
Sakshi Rane	123B1B242
Riya Borade	123B1B247

Under The Guidance of

Prof. Kapil Tajane



DEPARTMENT OF COMPUTER ENGINEERING, PIMPRI CHINCHWAD COLLEGE OF ENGINEERING, SECTOR 26, NIGDI, PRADHIKARAN, PUNE

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the mini project entitled "**MediConnect: Distributed Pharmacy Procurement and Multi-Distributor Medicine Supply Network**" has been carried out as a mini project for Distributed Systems Laboratory and submitted by the following students of Pimpri Chinchwad College of Engineering, Nigdi, Pune, in partial fulfillment of the requirements for B. Tech. (Computer Engineering).

Under the guidance of Prof. Kapil Tajane

In the partial fulfillment of the requirements for the B. Tech. (Computer Engineering)

Name	**PRN**
Sakshi Rane	123B1B242
Riya Borade	123B1B247

Prof. Kapil Tajane

Project Guide

ABSTRACT

Pharmacies frequently procure medicines from several distributors, each maintaining separate inventory, pricing and order records. A centralized inventory server cannot directly represent the independence of these suppliers, while manually contacting every distributor makes comparison and order processing inefficient. MediConnect is a distributed pharmacy procurement platform that connects pharmacy clients to multiple autonomous distributor services through a central FastAPI application programming interface (Main API). Each distributor runs its own gRPC server and SQLite database. The Main API discovers distributor nodes dynamically, queries them concurrently, aggregates medicine availability and prices, and coordinates medicine requirements and procurement orders. The implementation combines HTTP/REST, Protocol Buffers and gRPC, RabbitMQ event messaging, WebSockets for live status updates and WebRTC data channels for direct text chat. Docker Compose runs the components as separate containers; an optional fourth distributor can register without modifying the Main API. To demonstrate synchronization topics, the project maintains Lamport logical clocks, a simplified hub-observed vector, Cristian-style clock-offset estimates, a centrally simulated Bully election and a Chandy–Lamport-inspired distributed state capture. Atomic SQL updates prevent overselling and conflicting requirement claims. Three independent storage containers labelled Pune, Mumbai and Delhi replicate uploaded requirement images with read fallback. The project demonstrates partial-failure handling and scalable service discovery while acknowledging that its election, vector-clock and snapshot features are educational simplifications rather than complete decentralized algorithms.

Keywords: Distributed Systems, Pharmacy Procurement, FastAPI, gRPC, RabbitMQ, WebSocket, WebRTC, Lamport Clock, Bully Election, Distributed Storage, Docker.

INDEX

Chapter	Contents	Page No.
1	Introduction	5
	a) Problem Statement	5
	b) Project Idea	5
	c) Motivation	5
	d) Scope	5
	e) Literature Survey / Requirement Analysis	5
2	Project Design	8
	a) Hardware, Software and Resource Requirements	8
	b) Dataset and Database Design	8
	c) Hours Estimation	9
3	Module Description	10
	a) Block Diagram with Module Explanation	10
	b) Main Application Workflow	11
	c) Synchronization and Fault-Tolerance Modules	12
	d) Comparative Study	12
4	Results and Discussion	14
	a) Source Code and Execution	14
	b) Screenshots including GUI	14
	c) Test Cases	19
	d) Observations and Limitations	20
5	Conclusion and Future Scope	21
—	References	21

LIST OF FIGURES

Figure	Title	Page No.
3.1	System Block Diagram of MediConnect	10
3.2	Raft Node Role Transitions (requires correction)	10
4.1	Pharmacy Dashboard and Medicine Search Results	14
4.2	Distributor Dashboard and Inventory	15
4.3	Requirement List and Supplier Response Comparison	15
4.4	Order Lifecycle and Real-Time Status	16
4.5	Network Status: Distributor Health and Clocks	16
4.6	Bully Election and Distributed Event Log	17
4.7	Global Snapshot and Distributed Event Monitoring	17
4.8	Distributed Storage Region Status	18
4.9	WebRTC Text Chat	18
4.10	Docker Compose Running Services	19

1. Introduction

1.1 Problem Statement

Medicine procurement is fragmented across multiple independent distributors. Pharmacies need to check stock and prices, issue requirements, receive quotations, place orders and track fulfillment. Direct communication with each supplier is repetitive, and one unavailable distributor should not stop searches against other suppliers. The project addresses the need for one interface that coordinates multiple autonomous distributor services while preserving their independent databases and handling concurrent orders safely.

1.2 Project Idea

MediConnect provides two procurement workflows: (1) Find Suppliers, which searches a medicine across online distributor nodes in parallel; and (2) Requirement Lists, where a pharmacy posts multiple medicine needs and distributors independently evaluate and respond. A pharmacy compares offers, places an order and follows approval, dispatch and receipt. Direct text chat, real-time updates and replicated requirement images support communication. The system begins with three distributor nodes and can add a fourth through service self-registration.

1.3 Motivation

• Reduce manual supplier enquiries by aggregating multiple distributor responses.

• Demonstrate real distributed communication through autonomous services and separate databases.

• Support partial failure: a failed distributor should not invalidate successful responses from other distributors.

• Prevent inconsistent stock changes under simultaneous requests.

• Connect syllabus concepts to visible, testable features in a practical application.

1.4 Scope

In scope: pharmacy and distributor interfaces; authentication and dashboards; medicine search; requirement creation and response comparison; order lifecycle; gRPC and RabbitMQ; WebSockets; WebRTC text chat; health checks and dynamic registration; synchronization demonstrations; replicated requirement-image storage; Docker Compose deployment.

Out of scope: payment processing, production-grade multi-region hosting, Kubernetes deployment, Hadoop analytics, actual serverless hosting, blockchain, complete peer-to-peer Bully messaging, full distributed vector clocks and full Chandy–Lamport marker/channel-state capture.

1.5 Literature Survey and Requirement Analysis

Literature Survey

Distributed Systems Architecture: Tanenbaum and Van Steen [1] explain distributed architectures, scalability and fault tolerance, forming the foundation of MediConnect's multi-service architecture.

Remote Procedure Call (RPC): Birrell and Nelson [2] introduced RPC. MediConnect uses gRPC and Protocol Buffers [3] for communication between the Main API and distributors.

Message-Oriented Communication: RabbitMQ [4] enables asynchronous communication. MediConnect uses it to handle order events and notifications.

Logical and Vector Clocks: Lamport [5], Fidge [6] and Mattern [7] introduced techniques for tracking event ordering and causality. MediConnect demonstrates Lamport clocks and simplified vector clocks.

Leader Election: Garcia-Molina [8] proposed the Bully Algorithm. MediConnect uses a simplified version to select the highest-ranked available distributor.

Distributed Snapshots: Chandy and Lamport [9] introduced consistent global snapshots. MediConnect implements an inspired, simplified mechanism for collecting system states.

Distributed File Storage: Shvachko et al. [10] describe HDFS and data replication. MediConnect replicates uploaded images across three storage nodes.

Containerization: Docker [11] supports containerized deployment. MediConnect uses Docker Compose to run its independent services.

Requirement analysis

ID	Functional Requirement
FR1	Pharmacy and distributor staff can access role-specific dashboards.
FR2	Search medicine availability and prices across registered distributors.
FR3	Create requirement lists and receive independent distributor responses.
FR4	Place and track orders through approval, dispatch and receipt.
FR5	Deliver event notifications and real-time status updates.
FR6	Support direct pharmacy–distributor WebRTC text chat.

FR7	Detect distributor failure and continue querying available nodes.
FR8	Register an additional distributor without changing Main API code.
FR9	Display clocks, election state, event logs and snapshots.
FR10	Replicate uploaded requirement images across storage services.

ID	**Non-functional Requirement**
NFR1	Service isolation: each distributor owns its SQLite database.
NFR2	Concurrency safety: atomic stock and requirement-status updates.
NFR3	Resilience: timeouts and partial results during distributor failures.
NFR4	Maintainability: separate API, distributor, frontend, messaging and storage modules.
NFR5	Portability: local multi-container deployment with Docker Compose.

2. Project Design

2.1 Hardware, Software and Resource Requirements

Component	Technology / Purpose
Frontend	HTML, CSS and JavaScript; role-based dashboards
Main API	Python FastAPI; HTTP/REST, orchestration and aggregation
Distributor nodes	Python gRPC services; independent medicine inventory
Data storage	SQLite databases for individual distributor nodes and application data
RPC contract	Protocol Buffers: proto/pharmacy.proto
Messaging	RabbitMQ and notification consumer
Real-time communication	WebSockets for updates; WebRTC RTCDataChannel for text chat
Distributed storage	Three storage-service containers and replicated file metadata
Deployment	Docker and Docker Compose
Developer tools	Python, Git, code editor and modern web browser

Suggested development hardware: laptop/desktop capable of running Docker, with approximately 8 GB RAM or more and sufficient disk space for images and containers. These are practical recommendations, not measured minimum requirements.

2.2 Dataset and Database Design

The application uses seeded demonstration data rather than a published external dataset. Each distributor has a separate medicine inventory. The Main API stores pharmacy-related application data, orders and requirement metadata. The storage subsystem records image replicas and their regions.

Data Entity	Representative Fields / Purpose	Location
Distributor registration	Distributor ID, name, gRPC endpoint, heartbeat	Main API registry (in memory)

Medicine	Medicine ID, name, price, available quantity	Individual distributor SQLite database
Requirement	Pharmacy, requested medicines, quantities, status	Main API application database
Distributor response	Distributor, quoted availability/price, response status	Main API application database
Order	Pharmacy, distributor, items, quantity, lifecycle status	Main API / owning distributor workflow
Logical clock state	Main API counter, distributor counters, hub-observed vector	Main API and distributor processes
Uploaded image metadata	File identifier and storage replica regions	Main API metadata; files in storage services

2.3 Hours Estimation

The following is an illustrative effort plan for report formatting; replace estimates with actual team effort before submission.

Sr. No.	Activity	Estimated Hours
1	Requirement analysis and architecture	8
2	Frontend and dashboards	15
3	Main API and authentication	14
4	Distributor gRPC services and data	14
5	Requirements and order workflows	12
6	RabbitMQ, WebSocket and WebRTC integration	12
7	Synchronization demonstrations	10
8	Distributed file storage	8
9	Docker, testing and documentation	12
	Total	**105**

3. Module Description

3.1 Block Diagram and Module Explanation

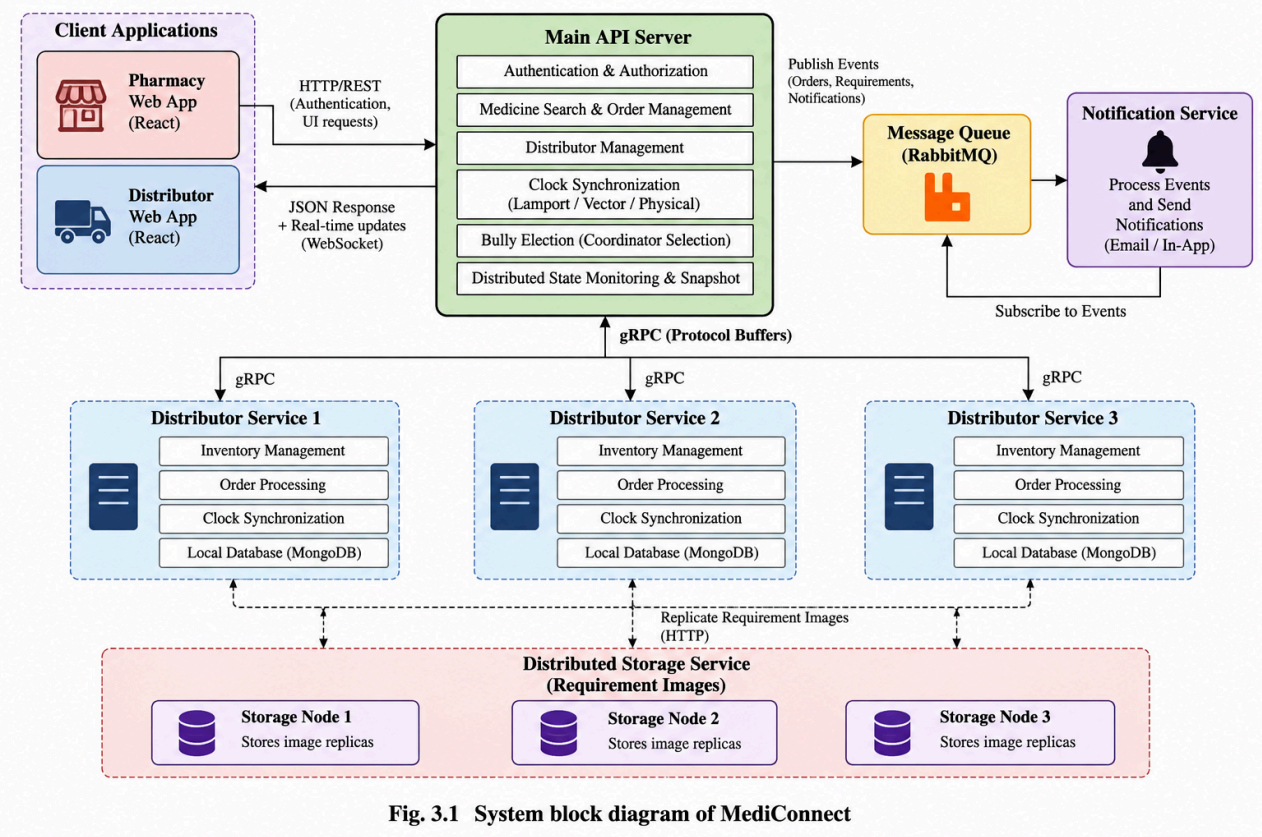


Fig. 3.1 System block diagram of MediConnect

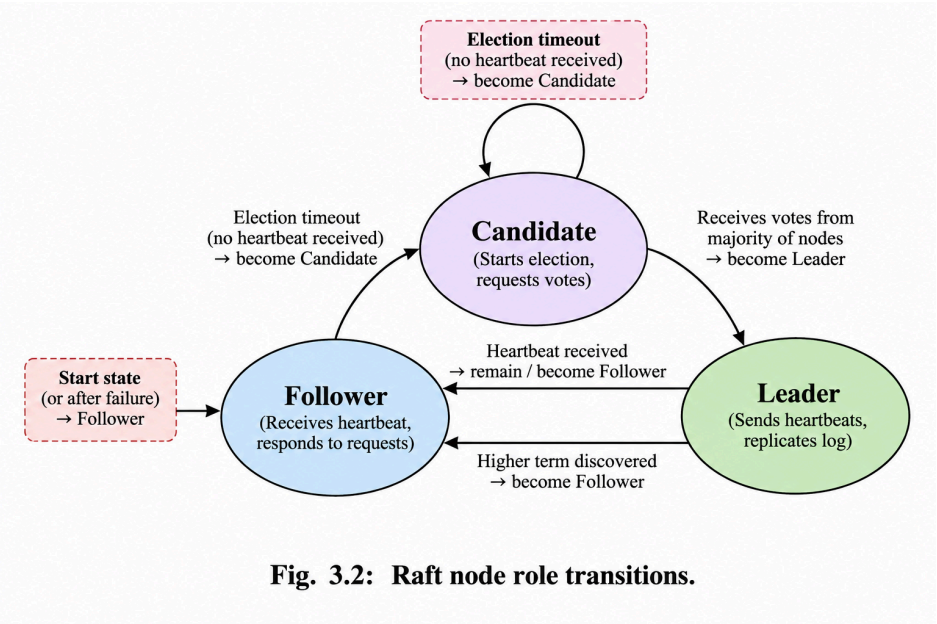


Fig. 3.2: Raft node role transitions.

Backend modules

Module	Code Location	Responsibility
Frontend	frontend/	Login, pharmacy/distributor dashboards, network status, chat
Main API	main_api/	REST endpoints, workflow management, RPC aggregation
Distributor gRPC	distributor_service/app/	Search, medicine details, order placement, stock update, health
gRPC definitions	proto/pharmacy.proto	Typed remote service contract
Service registry	main_api/app/grpc_clients/registry.py	Tracks distributor endpoints and registration
RPC client	main_api/app/grpc_clients/distributor_client.py	Concurrent calls, timeouts, partial results
RabbitMQ publisher	main_api/app/mq/publisher.py	Publishes business events
Notification consumer	notification_service/app/consumer.py	Processes events asynchronously
WebRTC signaling	main_api/app/ws/webrtc_signaling.py	Helps browsers establish data channel
Synchronization	main_api/app/sync/	Clock state, election and snapshot
Distributed storage	main_api/app/dfs/storage.py; storage_service/	Image replication and fallback reads
Deployment	docker-compose.yml	Starts and networks services

3.2 Main Application Workflow

1. A pharmacy logs in and searches for a medicine.

2. The Main API finds registered distributor endpoints and sends parallel gRPC SearchMedicine requests.

3. Each distributor queries its own SQLite inventory and returns its result.

4. The Main API aggregates available results and displays price/stock comparison.

5. A pharmacy can publish a multi-item requirement; RabbitMQ carries a requirement_created event.

6. Distributors evaluate requirements and submit offers; WebSockets push updates to the browser.

7. A pharmacy places an order; distributor approval uses an atomic stock decrement.

8. Order dispatch/receipt changes are published as events; pharmacy inventory is updated as appropriate.

9. Users can exchange direct text messages through a WebRTC data channel.

3.3 Synchronization and Fault-Tolerance Modules

Concept	Implementation	Accuracy / Limitation
Lamport logical clocks	Independent counters at Main API and distributors; piggyback on selected gRPC calls	Uses max(local, received)+1
Vector clock display	Main API observes per-node logical values	Hub-observed vector, not full causal vector propagation
Physical clock estimate	HealthCheck wall-clock timestamp and RTT	Cristian-style estimate; does not set OS clocks
Bully election	Highest online distributor ID selected; re-election on failure	Main API simulates election trace, not peer-to-peer messages
Mutual exclusion	Atomic SQL stock and requirement-claim updates	Database concurrency safety, not Ricart–Agrawala
Global snapshot	Collects Main API and distributor state	Inspired by global-state capture; no MARKER/in-flight channel tracking
Fault tolerance	Per-node RPC timeouts and partial aggregation	Main API remains a central dependency
Storage replication	Copy images to three independently running storage nodes	City labels simulated on one host; no auto-repair

3.4 Comparative Study

Alternative	Chosen Approach	Reason
One centralized distributor inventory	Separate distributor gRPC services	Represents independent suppliers and databases

Sequential distributor requests	Parallel gRPC fan-out	Avoids waiting for each node in sequence
Only synchronous notifications	RabbitMQ events	Separates event production from consumption
Polling-only browser updates	WebSockets	Immediate updates without repeated refresh
Ring election	Simplified Bully selection	Registered IDs and central health visibility; no ring topology
Full distributed mutual-exclusion protocol	Atomic SQLite guards	Correctly protects stock at the single owning database
Hadoop / HDFS	Small replicated storage service	Files are small; no large-scale block processing required
Kubernetes / cloud hosting	Docker Compose locally	Sufficient for a multi-service educational prototype

4. Results and Discussion

4.1 Source Code and Execution

Source code is provided in the MediConnect project archive. A public GitHub repository URL has not been verified; insert the actual repository link if published.

Deployment steps: (1) open the extracted mediconnect_v2_final folder; (2) run docker compose up --build; (3) run python seed_demo_data.py in another terminal. Frontend: http://localhost:8080; API docs: http://localhost:8000/docs; Network Status: http://localhost:8080/network-status.html; RabbitMQ UI: http://localhost:15672. An optional fourth distributor can be started with docker compose --profile scale up -d distributor-d.

The features below are supported by the project source and documentation. Screenshots and actual test outcomes must be captured from a running instance before final submission; they are not fabricated in this report.

4.2 Screenshots Including GUI

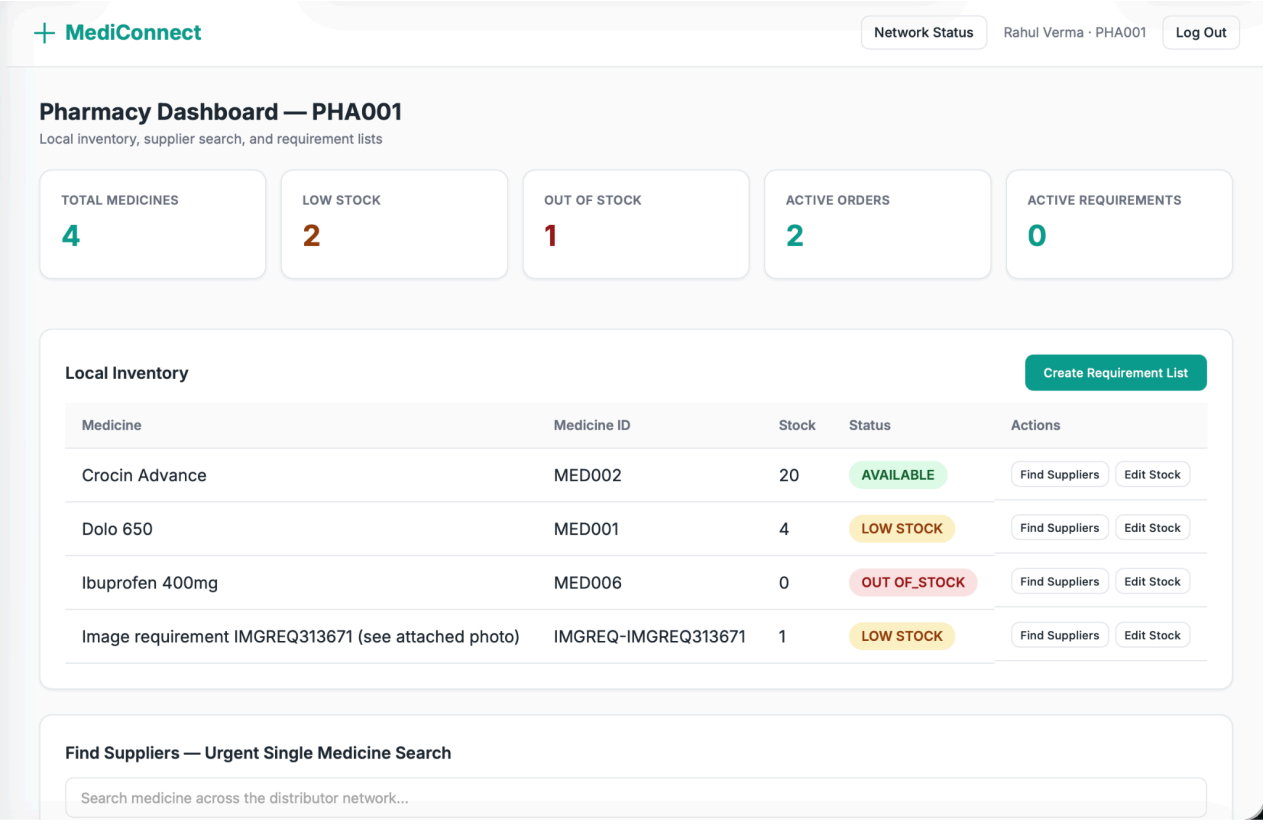


Figure 4.1: Pharmacy dashboard and medicine search results

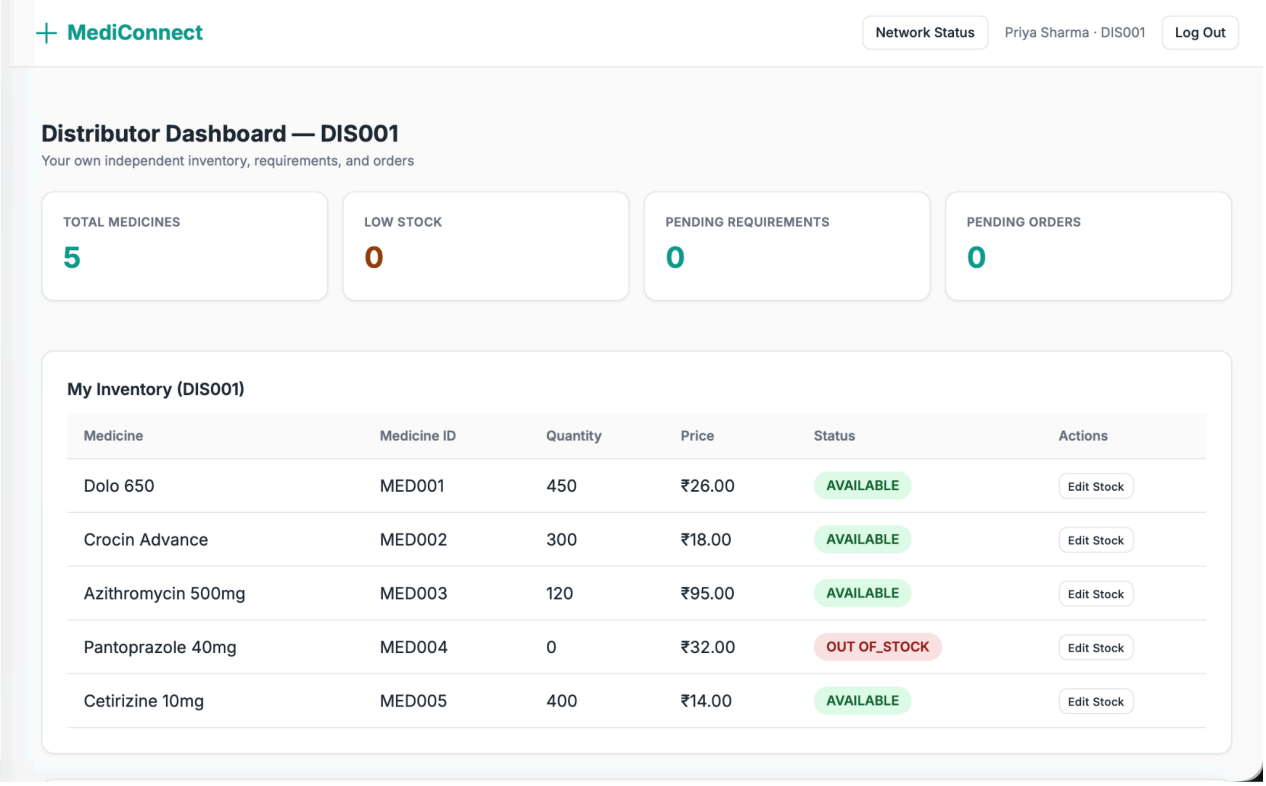


Figure 4.2: Distributor dashboard and inventory

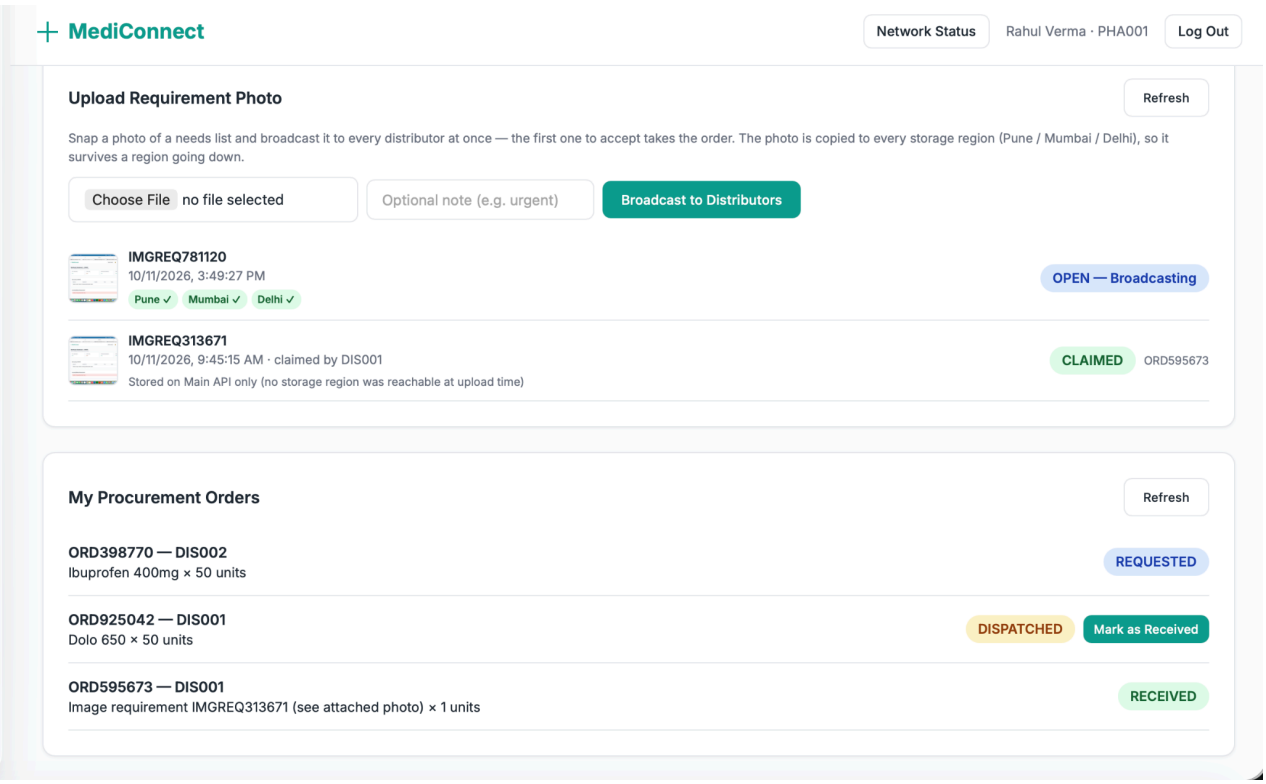


Figure 4.3: Requirement list and supplier response comparison

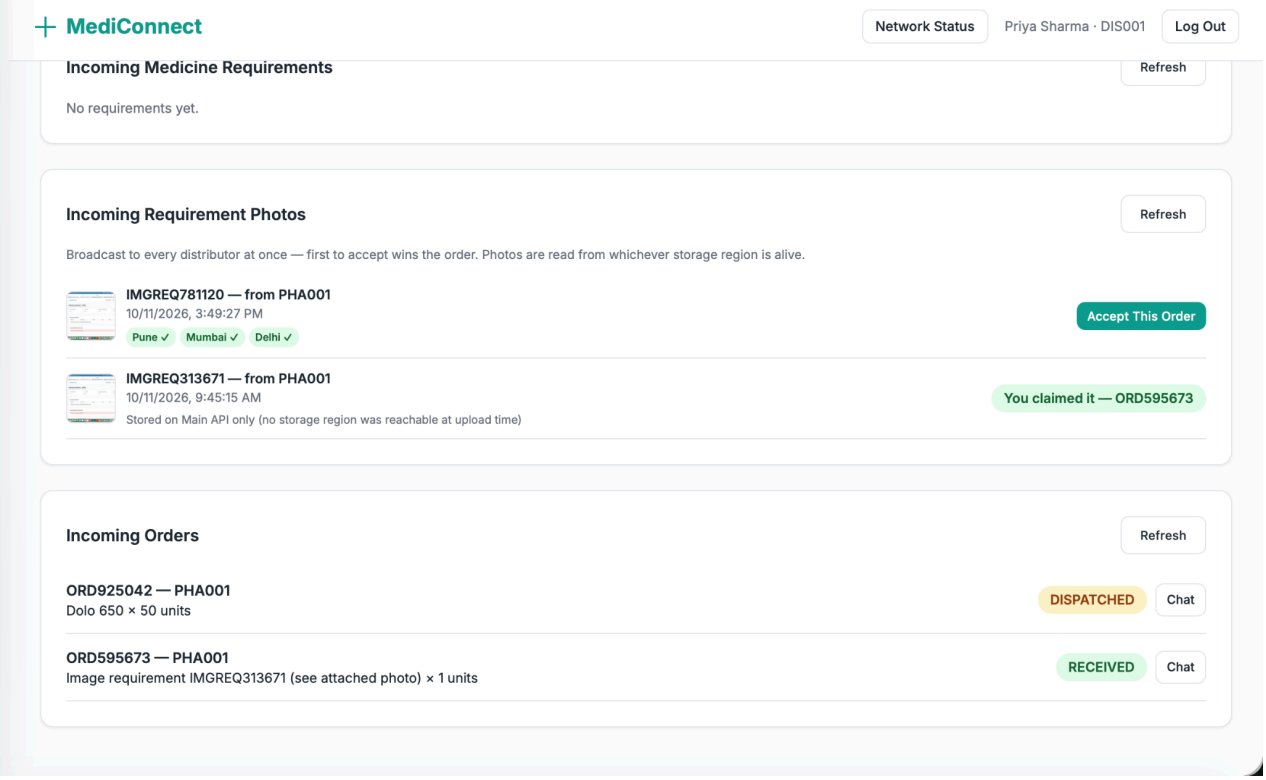


Figure 4.4: Order lifecycle / real-time status

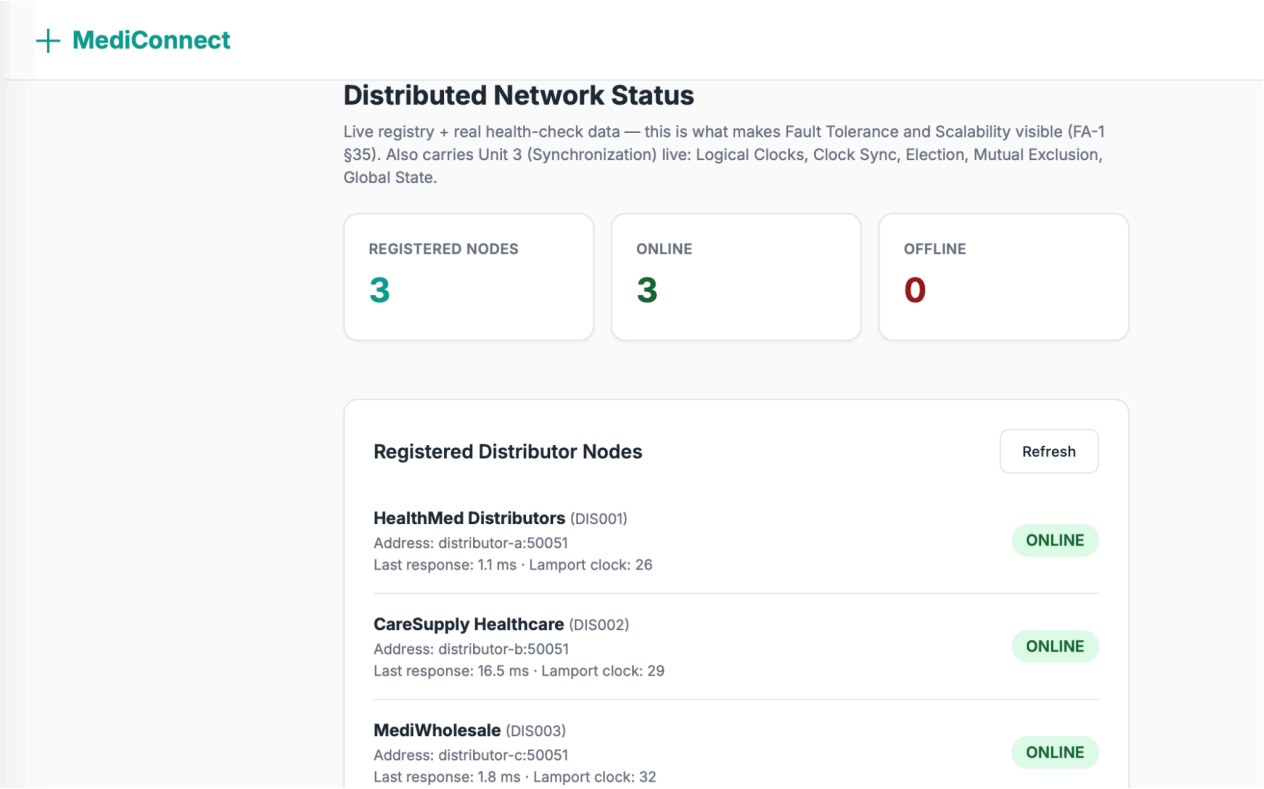


Figure 4.5: Network Status: distributor health and clocks

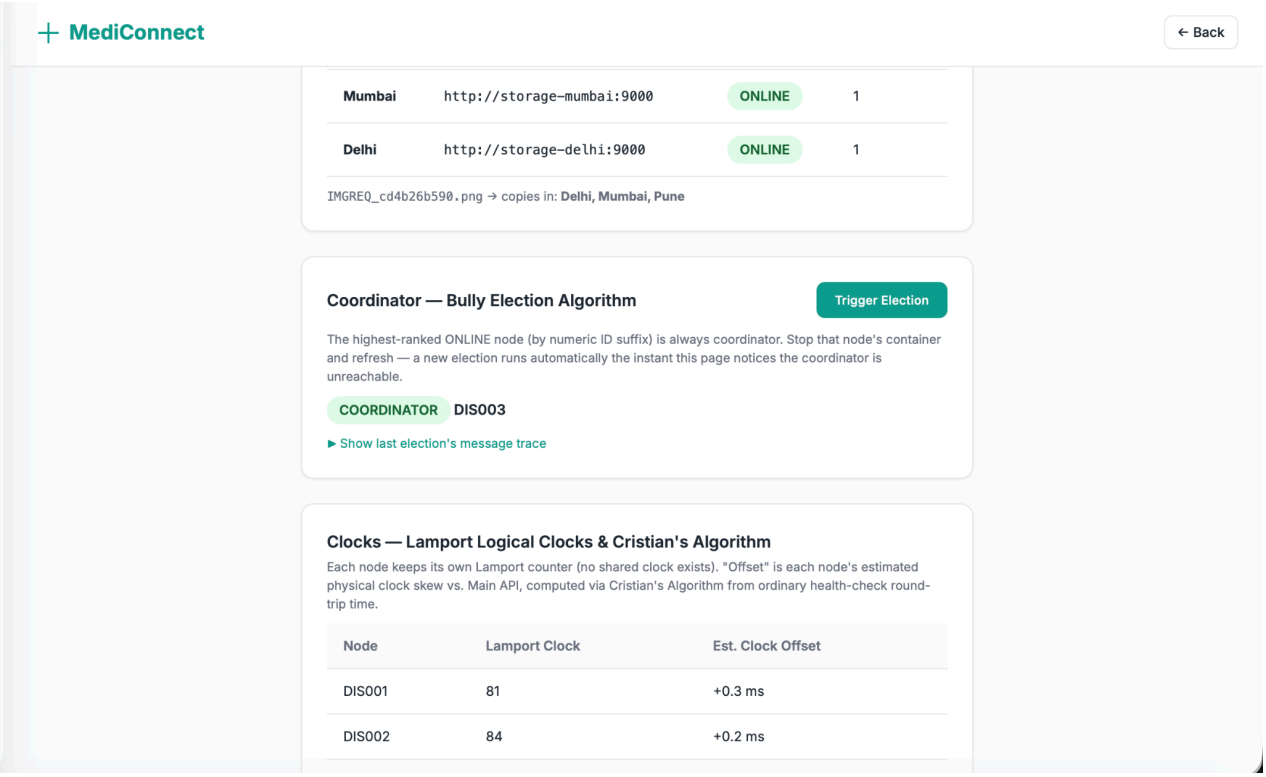


Figure 4.6: Bully election and distributed event log

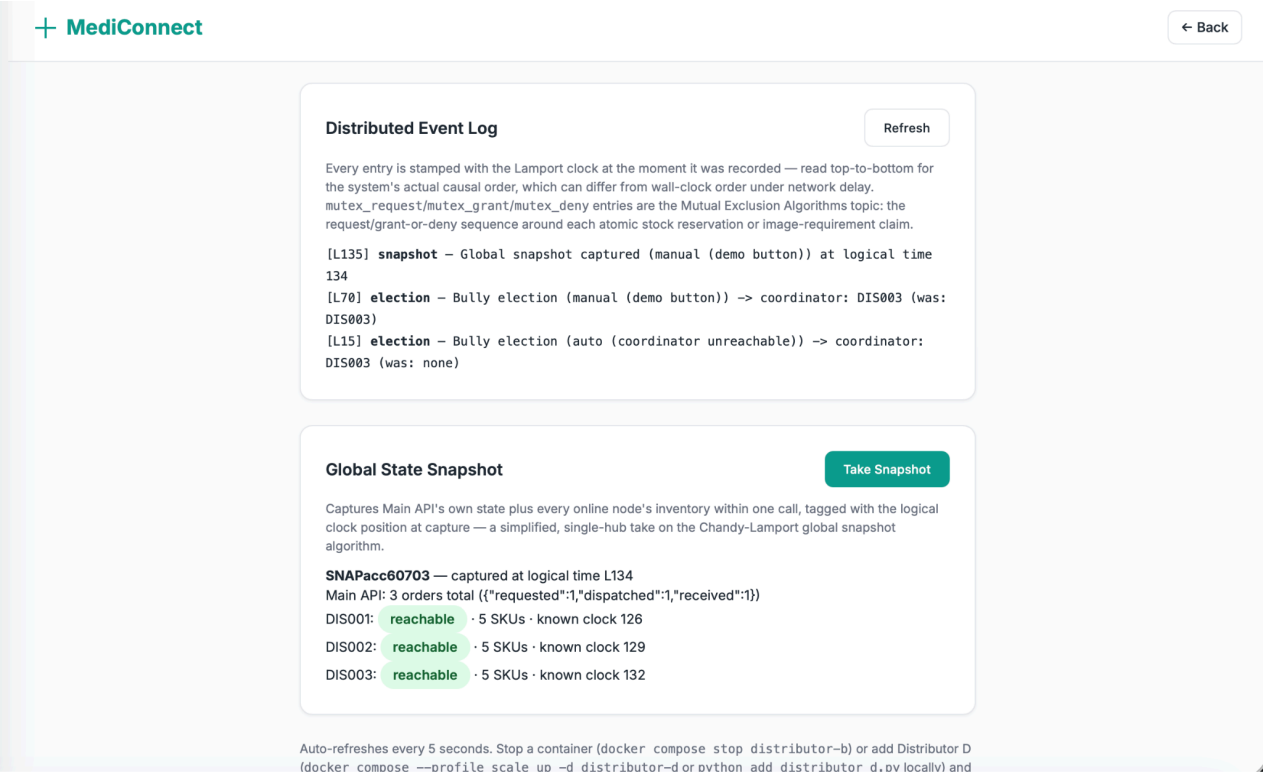


Figure 4.5: Global snapshot and storage-region status

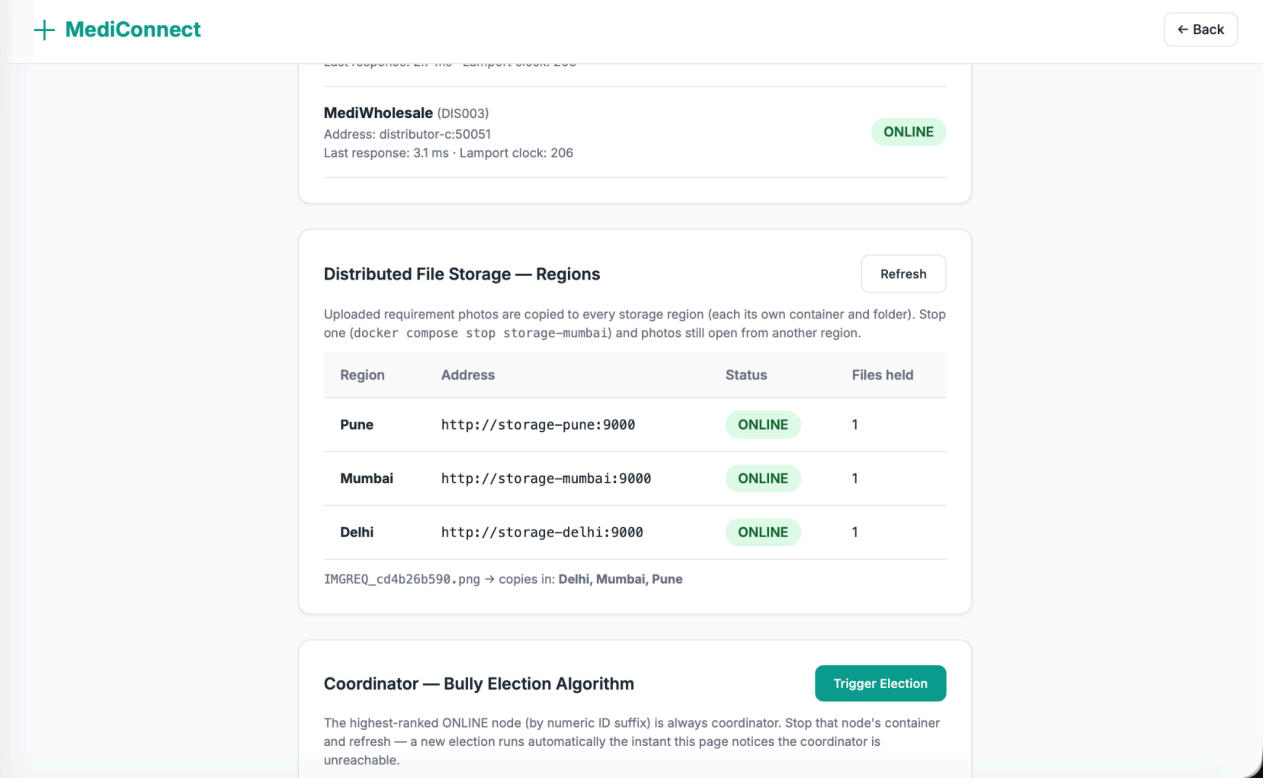


Figure 4.6: Storage-region status

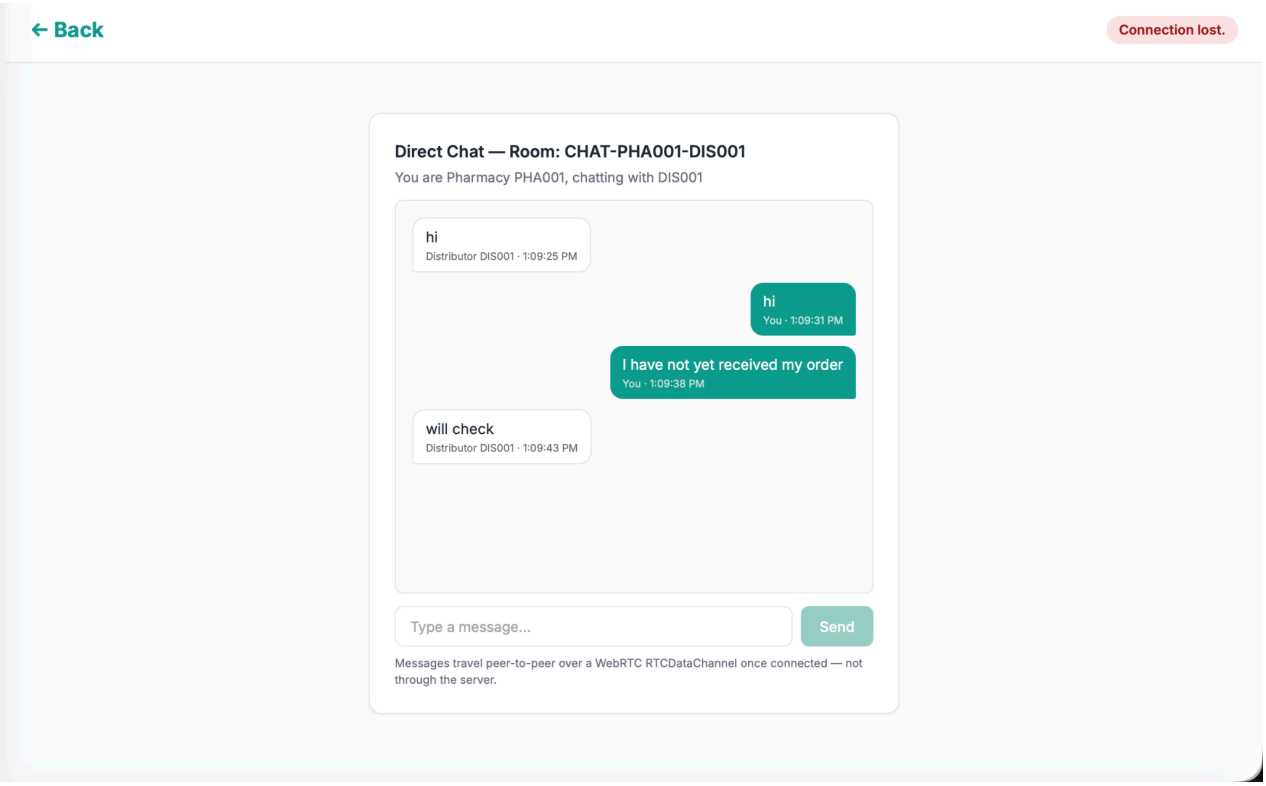


Figure 4.7: WebRTC text chat

```
sakshirane@192 mediconnect_v2_final % docker compose up --build
WARN[0000] /Users/sakshirane/Downloads/mediconnect_v2_final/docker-compose.yml: the attribute `ver
sion` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 2.3s (57/57) FINISHED
 => [internal] load local bake definitions                                          0.0s
 => => reading from stdin 4.86kB                                                    0.0s
 => [distributor-a internal] load build definition from Dockerfile                  0.0s
 => => transferring dockerfile: 725B                                                0.0s
 => [storage-mumbai internal] load build definition from Dockerfile                 0.0s
 => => transferring dockerfile: 303B                                                0.0s
 => [main-api internal] load build definition from Dockerfile                       0.0s
 => => transferring dockerfile: 730B                                                0.0s
 => [frontend internal] load build definition from Dockerfile                       0.0s
 => => transferring dockerfile: 157B                                                0.0s
 => [notification-service internal] load build definition from Dockerfile           0.0s
 => => transferring dockerfile: 245B                                                0.0s
 => [frontend internal] load metadata for docker.io/library/nginx:alpine            1.3s
 => [storage-pune internal] load metadata for docker.io/library/python:3.11-slim    1.6s
 => CACHED [notification-service internal] load .dockerignore                       0.0s
 => => transferring context: 2B                                                     0.0s
 => [frontend 1/3] FROM docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5a  0.0s
 => => resolve docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf805455  0.0s
 => [frontend internal] load build context                                          0.0s
 => => transferring context: 96.11kB                                                0.0s
 => CACHED [frontend 2/3] COPY frontend/ /usr/share/nginx/html/                     0.0s
 => CACHED [frontend 3/3] COPY docker/nginx.conf /etc/nginx/conf.d/default.conf     0.0s
 => [frontend] exporting to image                                                   0.0s
 => => exporting layers                                                             0.0s
 => => exporting manifest sha256:8688ef46c7afb160159502f425f37896eaa139b57fcca43606a2f1a842  0.0s
 => => exporting config sha256:3657b6cdd12ee9f085cdbc6c1fa94ea6d9bd72b6243486c3669696ea3ddd  0.0s
 => => exporting attestation manifest sha256:8995e9d2804c072f88f20d912e133596f7e020a8a7e9f3  0.0s
 => => exporting manifest list sha256:d69cbd472c808031bd24fa8444a79692337712ed9d172308ae480  0.0s
 => => naming to docker.io/library/mediconnect_v2_final-frontend:latest             0.0s
 => => unpacking to docker.io/library/mediconnect_v2_final-frontend:latest          0.0s
 => [frontend] resolving provenance for metadata file                               0.0s
 => [storage-mumbai internal] load build context                                    0.0s
 => => transferring context: 222B                                                   0.0s
```

Figure 4.8: Docker Compose running services

4.3 Test Cases

ID	Scenario	Expected Result	Execution Status
TC1	Search a medicine with all 3 distributors online	Responses aggregated from available nodes	To verify
TC2	Stop distributor B, then search	A/C results still appear; B unavailable	To verify
TC3	Restart distributor B	Node re-registers / returns online	To verify
TC4	Add distributor D via scale profile	Registry grows from 3 to 4	To verify
TC5	Submit requirement and distributor response	Status updates and event published	To verify
TC6	Approve concurrent orders exceeding stock	Atomic guard prevents overselling	To verify

TC7	Two distributors claim same open requirement	Only one atomic claim succeeds	To verify
TC8	Open WebRTC chat in two roles	Text exchange through RTCDataChannel	To verify
TC9	Take distributed state snapshot	Main API and reachable distributor states shown	To verify
TC10	Stop current elected coordinator	Next-highest online ID selected	To verify
TC11	Upload requirement image	Replica metadata and copies stored	To verify
TC12	Stop one storage region; retrieve existing photo	Read succeeds through surviving replica	To verify

4.4 Observations and Limitations

• The Main API hides individual distributor addresses from pharmacy users and combines parallel responses.

• Each distributor owns its own SQLite data, which preserves supplier independence.

• Partial distributor failure is handled with timeouts; however, failure of the Main API can interrupt the overall service.

• The distributor registry is held in memory and is repopulated through heartbeats after a Main API restart.

• Clock synchronization estimates offsets but does not correct physical clocks.

• The election is centrally simulated; the snapshot does not guarantee a consistent global cut.

• The replicated storage nodes are on the same Docker host and therefore do not provide geographic disaster recovery.

• Runtime test results, screenshots and quantitative performance metrics must be added after actual execution.

5. Conclusion and Future Scope

MediConnect demonstrates how independent distributor services can be integrated into a unified pharmacy procurement workflow without merging their underlying inventories. A central FastAPI middleware service discovers suppliers and coordinates parallel gRPC queries, while RabbitMQ supports asynchronous events, WebSockets deliver live updates and WebRTC enables direct text chat. Docker Compose makes the architecture reproducible, and atomic database operations protect limited stock during concurrent ordering. Lamport clocks, clock-offset estimation, Bully-style election and global-state capture make important Distributed Systems concepts visible through a Network Status dashboard. A simplified replicated image-storage layer extends the application into distributed file storage.

Future scope includes production authentication and access controls; persistent, highly available service discovery; independent Main API replicas; deployment to multiple hosts or cloud infrastructure; Kubernetes orchestration; monitoring and tracing; stronger transactional workflows; repair and consistency checks for file replicas; and full peer-to-peer election, vector-clock and Chandy–Lamport implementations where educational or operational requirements justify them.

References

[1] A. S. Tanenbaum and M. van Steen, Distributed Systems, 3rd ed., 2017.

[2] G. Coulouris et al., Distributed Systems: Concepts and Design, 5th ed., 2011.

[3] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” Communications of the ACM, 1978.

[4] K. M. Chandy and L. Lamport, “Distributed snapshots: determining global states of distributed systems,” ACM TOCS, 1985.

[5] H. Garcia-Molina, “Elections in a distributed computing system,” IEEE Transactions on Computers, 1982.

[6] FastAPI Documentation, https://fastapi.tiangolo.com/

[7] gRPC Documentation, https://grpc.io/docs/

[8] RabbitMQ Documentation, https://www.rabbitmq.com/docs

[9] Docker Compose Documentation, https://docs.docker.com/compose/

[10] MDN Web Docs, WebRTC API, https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API

[11] MediConnect project README.md and UNIT3_UNIT4_MAPPING.md (provided project archive).